Introduction

Alpha-carboxysomes are a bacterial microcompartment responsible for carbon fixation in some chemoautotrophs and cyanobacteria. They are critical for their hosts' survival, but they can also be expressed and function in other bacteria and plant cells, and even function in vitro (outside of a cell). We have crystal structures of many of their proteins, but there are many unanswered questions about how the compartment is organized.

We performed cryo electron tomography with subtomogram averaging to look at the major enzyme cargo, Rubisco. We found that it polymerizes into long chains at high concentrations, and that those chains can arrange into a lattice.

This script package processes the tomogram data and performs biophysical analyses on volumes, concentrations, binding partners, any conformational changes, interaction sites and more to understand how all of this works!

We make this software package publicly available for others who may be interested in researching properties of Rubisco or the polymerization of any other proteins of similar features that can be adapted to run with these scripts.

Installation

This script package requires MATLAB and a Dynamo installation (www.dynamo-em.org). The version used in this current toolkit was 1.1.532. It will run on any operating system and hardware configuration.

The script package is adapted from a precursor collection of ObservableHQ (JavaScript) modules that were translated to MATLAB and then further developed (Metskas et al, *Nature Communications* 2022: Rubisco forms a lattice inside alpha-carboxysomes). The new MATLAB script package is hosted online in GitHub for version control. To compile from the source using git, follow the steps outlined below:

- git clone https://github.com/LAMetskas/2025_polymerizationAnalysis.git
- cd MetskasLab
- run MATLAB executable
- In MATLAB command window: run <DYNAMO_ROOT>/dynamo_activate.m
- Run scripts using the command window

We use Dynamo to read in the tomogram data from a '.tbl' file. Dynamo is a Matlab based set of scripts and GUIs designed to perform subtomogram averaging from cryo-electron tomograms. Dynamo commands must be activated by running the command in step 4 before starting.

Operation

Input Data Format

Convex_hull_and_volume.m will read in the input data from a Dynamo table file (extension '.tbl'). This file comes from a completed subtomogram averaging project with at least subnanometer resolution and high-accuracy particle picking.

For users who prefer a different software package for subtomogram averaging, the project output must be converted to Dynamo format prior to running this script package. The format of the file is the following, and is explained in more detail in the Dynamo documentation (www.dynamo-em.org).

1 row per particle with 31 columns/fields of data.

Columns:
- Tag – particle ID number.
- Aligned
- Averaged
- dx - Shift in the x direction. If non-zero, this will be added to the x position (col. 24)
- dy - Shift in the y direction. If non-zero, this will be added to the y position (col. 25)
- dz - Shift in the z direction. If non-zero, this will be added to the z position (col. 26)
- tdrot (Euler angle Z)
- tilt (Euler angle X)
- narot (Euler angle Z')
- cc
- cc2
- cpu
- ftype
- ymintilt
- ymaxtilt
- xmintilt
- xmaxtilt
- fs1
- fs2
- tomo – The tomogram number from which this particle was obtained
- reg – The ID number of the Carboxysome that this Rubisco is in.
- class
- annotation
- x - the position of this particle in the x axis
- y - the position of this particle in the y axis
- z - the position of this particle in the z axis
- dshift
- daxis
- dnarot
- dcc
- otag
- npar
- ref
- sref
- apix
- def
- eig1
- eig2

Main Pipeline Scripts
The *main.m* script calls the main pipeline scripts in order and ports information between them. These scripts are:

- convex_hull_and_volume.m
- local_global_alignment.m
- linkagesNewNew.m
- chain_maker.m
- chain_linkages.m
- lattice_gen.m

Graphing & Analysis Scripts
In all descriptions, "ROI" refers to the volume containing the particles of interest (in our use case, a carboxysome); "particle" refers to the protein targeted for subtomogram averaging (in our use case, Rubisco).

General ROI scripts:
- concentration_volume_graphs
  - Generates graphs of 1) volume of ROI vs. the number of outer-layer particles and 2) concentration of particles within ROI vs. the number of outer-layer particles
- angle_with_outer_shell
  - Generates a histogram of angles between the ROI boundary and the outer-layer particles. This graph is overlaid by a calculation of random angles for comparison.
- tensor_analysis_graphs
  - Generates graphs of 1) scalar orientation parameter S as a function of ROI total concentration and 2) S as a function of ROI inner particle concentration.
- neighboring_rubisco_alignment_analysis
  - Generates probability density graphs of the angles between particle Z' axis vectors within a user-specified distance of each other. The graphs are grouped by ROI. A set of randomly generated angles is stacked on top of the real data for reference.
- create_class_table
  - Generates a csv file containing information about all particles in the dataset, including parent ROI concentration and inner concentration, ROI volume, whether the particle is part of a chain, whether its chain is at the center of a lattice, whether it is an outer-layer particle, a list of its oligomerization partners, and its spatial relation to its neighbors in its chain.

Polymerization analysis scripts:
- rubisco_alignment_analysis
  - Generates a graph of the number of "aligned" particles in each ROI as a function of ROI parameters. "Aligned" particles are those that are closer than a user-defined distance and whose orientation vectors make an angle smaller than a user-defined angle.
- affinity_by_sites
  - Generates graphs of the ratio of bound to unbound particle binding sites in a ROI, one as a function of the ROI's total particle concentration and the other as a function of its inner concentration. Also writes a csv file containing each ROI's number of bound and unbound sites, concentration, number of particles, and longest chain. It also fits an exponential function to the first graph and prints the resulting parameters to the matlab command window.
- neighboring_rubisco_bend_analysis
  - Generates a histogram of the bend angles between adjacent particles in a chain
- neighboring_rubisco_twist_analysis
  - Generates a histogram of the twist angles between adjacent particles in a chain

- twist_and_bend_analysis_graph
  - Generates a plot of the twist angle between two adjacent particles in a chain as a function of their bend angle
- rubisco_spacing_analysis
  - Generates a set of probability density graphs of the distance between adjacent particles in a chain, grouped by ROI. The distance is calculated along the source particle's axis.
- chain_lengths_analysis
  - Generates a histogram of the lengths of chains in the dataset. By setting a minimum chain length the user can filter out shorter chains. By setting a minimum number of linkages, the user can filter out chains which participate in few linkages with other chains.
- chain_spacing_analysis
  - Generates graphs concerning the spatial relationships between chains. 1) A histogram of the angles between chains within a user-specified distance of each other. 2) A set of probability density graphs of the distances between two chains, grouped by ROI.
- synthetic_data_generation
  - Generates two tables of synthetic data for validation purposes. 1) A particle table with randomized coordinates and euler angles. 2) A particle table with a known percent of particles that form linkages.

Visualization scripts:

- Visualize_Carboxysome_3D
  - Creates an interactive model showing particles in an ROI. The user can highlight specific particles and chains and control the colors of the scene.
- display_chain_tags
  - Prints the tags of all particles in chains in a user-specified ROI to the matlab command window. The formatting of the output is designed for easy use in the 3D Carboxysome Visualization.
- display_lattice_tags
  - Prints the tags of all particles in lattices in a user-specified ROI to the matlab command window. The formatting of the output is designed for easy use in the 3D Carboxysome Visualization.

Helper Scripts and Classes:

- Preprocess_datasets
  - Prepares a single input table from a Dynamo subtomogram averaging project. Concatenates even and odd half-sets into a single table, and combines xyz coordinates into a single set of coordinates in columns 24-26. Transforms all values in the table to real values.
- Constants
  - Defines the constants to be used by the scripts in the main pipeline. Must be changed depending on particle and dataset of interest.
- Read_data_tbl
  - Reads data from .tbl file and stores the output in an array
- Get_carboxysomes_indices_from_tomogram
  - Gets all the unique ROI indices from a data table
- Read_rubisco_objects_from_tomogram
  - Reads each line of a data table into a particle object
- Carboxysome
  - Model for ROI
- Rubisco

    - Model for storing data read from the .tbl file in a convenient way
- Rubisco_Pair
    - Model for a pair of particles
- DisjointSet
    - Used by chain_maker and lattice_gen to put together the particles that are linked together into chains and the chains that are linked together into lattices
- Rubisco_Chain
    - Makes an instance of a particle chain, including the tags in it
- Calc_bend
    - Calculates the bend angle between adjacent particles in a chain
- Calc_narot
    - Calculates the narot angle between adjacent rubiscos in a chain
- Calc_twist
    - Calculates the twist angle between adjacent rubiscos in a chain
- Chain_Pair
    - Makes an instance of a chain pair, same functionality as the particle pairs
- ChainType
    - Specifies whether chains are attached to the ROI boundary, growing along the ROI boundary, or free-floating
- LatticeType
    - Classifies lattices into full, incomplete, none, multiple, or edge type

Code of Conduct

This project adheres to the Contributor Covenant code of conduct. By participating, you are expected to uphold this code. Please report unacceptable behavior to metskas@purdue.edu

Reporting Issues

If you believe you have found an error, malpractice, or security issue, please responsibly disclose by contacting us at metskas@purdue.edu

Individual Program Descriptions

1. convex_hull_and_volume
   Calculate the volume of carboxysomes from a data set based on a convex hull calculation. Reads in particles from a data file and populates Carboxysome objects, their corresponding Rubisco objects, and computes their properties.

- **Syntax**
  `carboxysome_data = convex_hull_and_volume(filename)`

- **Description**
  `carboxysome_data = convex_hull_and_volume(filename)` reads the particle data from *filename* and returns an array of Carboxysome class objects with properties like volume, concentration, inner concentration, number of total rubisco, number of inner and outer rubisco, tomogram number, positions, convex hull, vertices, normals, centroids, and its corresponding array of Rubisco objects.

- **Examples**

```
>> input_file = 'DTT_r3bi2.tbl';
>> carb_volumes = convex_hull_and_volume(input_file)

carb_volumes =

 1x131 Carboxysome array with properties:

  carb_index
  rubisco
  num_rubisco
      num_rubisco_inner
      num_rubisco_outer
      tags_inside
      convex_hull
      vertices
      coordinates
      volume
      concentration
      inner_concentration
      num_events
      rubisco_pairs
      normals
      centroids
      S
      eigenvalues
      S_val
      ave_vector
      links
      chains
      chain_links
      lattice
      max_connections
      rings
```

```
      tomo
      lattice_type

>> carb_volumes(1)

ans =

Carboxysome object with properties:
  carb_index = 1
  rubisco = Rubisco array of length 184
  num_rubisco = 184
  num_rubisco_inner = 104
  num_rubisco_outer = 80
  tags_inside = [196 195 193 192 189 187 178 ...]
  convex_hull = [1 70 178;1 100 155;1 103 183;1 111 154;...]
  vertices = [381.2 1912.4 443.21;388.12 1927.7 488.74;...]
  coordinates = [674.46 1972.5 529.96;666.6 1944.9 502.94;...]
  volume = 3.1556e-22
  concentration = 968.2292
  inner_concentration = NaN
  num_events = NaN
  rubisco_pairs = Rubisco_Pair array of length 0
  normals = [0.422168122847842 0.302981011716375
0.85438667041946;...]
  centroids = [639.116666666667 1970.43333333333
548.156666666667;...]
  S = []
  eigenvalues = []
  S_val = NaN
  ave_vector = []
  links = Rubisco_Pair array of length 0
  chains = Rubisco_Chain array of length 0
  chain_links = Chain_Pair array of length 0
  lattice = Chain_Pair array of length 0
  max_connections = NaN
  rings = NaN
  tomo = 1
  lattice_type = LatticeType: none
```

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No

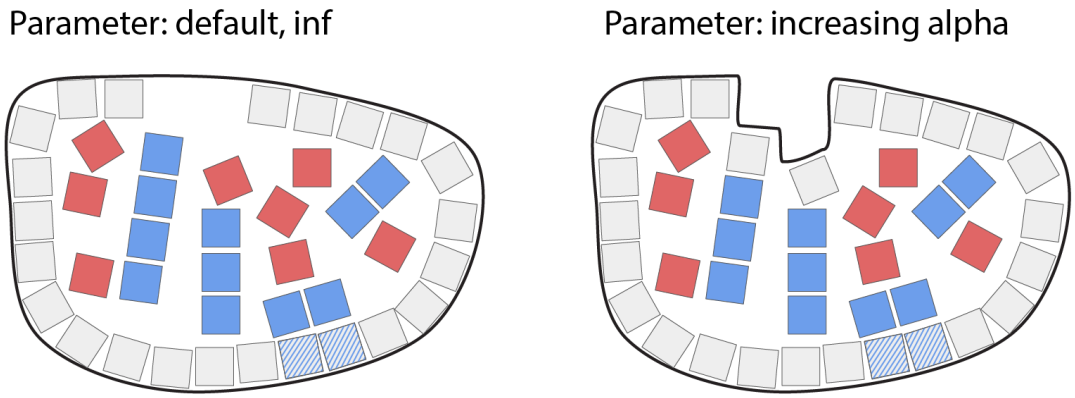
- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects populated with computed property values: volume, concentration, inner concentration, number of total rubisco, number of inner and outer rubisco, tomogram number, positions, convex hull, vertices, normals, centroids, and an array of all Rubisco objects that correspond to this carboxysome.

- **Additional Parameters**

When defining the perimeter of the ROI, we use the function `alphaShape` for a 3D convex hull, with the specific 'alpha' flag/parameter (the alpha radius) that can be adjusted to the needs of the user.

The default `alpha` = Inf. This returns a boundary A. below. However, some systems may need a tighter perimeter such as B. below. In this case the alpha should be changed.



In order to find what values of `alpha` are "critical" or make a difference in the formation of the shape, the user can use `alphaSpectrum` to view all the critical alpha values to use. This is a range where `alpha` = 0 would create an empty shape.

If the user would need to change the default, we recommend users to change values in the following manner:

```
[convex_hull, ~] = convhull(x, y, z); (Line 41 on GitHub)
```

Change to

```
shape = alphaShape(x, y, z, <yourAlphaParameterHere>);
convex_hull = boundaryFacets(shape);
```

The user can select what value they would like for `alpha`, using `alphaSpectrum` or the function `criticalAlpha` to help with selecting potential values. If they would want a default convex_hull, set `alpha` to Inf. Alternatively, the user could skip the `alpha` parameter and let it select a tight fit that includes all points by default.

2. local_global_alignment

Calculate the pairwise distance and angle between every pair of Rubiscos within each carboxysome. Implements Spatial Hashing to optimize and only consider neighboring pairs of Rubisco. Also calculates the global alignment of each carboxysome using tensor analysis.

- **Syntax**

```
  carboxysome_data = local_global_alignment(filename, carboxysomes,
max_distance)
  carboxysome_data = local_global_alignment(filename, carboxysomes)
  carboxysome_data = local_global_alignment(filename)
```

- **Description**

`carboxysome_data = local_global_alignment(filename, carboxysomes, max_distance)` takes in an array of Carboxysome objects with pre-computed properties from **convex_hull_and_volume** and populates each Carboxysome's rubisco_pairs property with Rubisco_Pair objects for every two rubisco in the carboxysome that are within max_distance of distance from each other. The pairs are only created in one direction. That is, if pair (1, 2) exists, then pair (2, 1) will not be created. In the returned Carboxysome array, properties num_events, ave_vector, S, eigenvalues, and S_val are also computed and populated.

`carboxysome_data = local_global_alignment(filename, carboxysomes)` takes in an array of Carboxysome objects with pre-computed properties from **convex_hull_and_volume** and populates each Carboxysome's rubisco_pairs property with Rubisco_Pair objects for every two rubisco in the carboxysome that are within 2*√3*rubisco_diameter of distance from each other (any two rubisco that fit into a box of side length 2 * rubisco_diameter). The pairs are only created in one direction. That is, if pair (1, 2) exists, then pair (2, 1) will not be created. In the returned Carboxysome array, properties num_events, ave_vector, S, eigenvalues, and S_val are also computed and populated.

`carboxysome_data = local_global_alignment(filename)` allows the user to provide only the *filename* as input and use this script as the starting point (without having to run other scripts first). If only one argument is provided, the script will call back to **convex_hull_and_volume** with the provided *filename* argument and generate the pre-required Carboxysome array before creating pairs.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysomes`	Array of Carboxysome objects for which to create pairs and compute alignment properties. Carboxysome objects must be pre-populated with carb_index, volume, concentration, num_rubisco, num_rubisco_inner, rubisco (array of Rubisco	Yes

	objects each with index, tag, position, vector, etc.), and tags_inside. This is the output array from **convex_hull_and_volume.** It is intended that the user run **convex_hull_and_volume** first and then run **local_global_alignment** with the obtained output as an argument. Alternatively, the user may run this script as the starting point by only providing the filename parameter, and the script will reference **convex_hull_and_volume** to obtain the required information.	
`max_distance`	The maximum distance to allow between a valid pair of rubiscos. Measured in pixels. By default this value is twice the diameter of a rubisco.	Yes

- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects (these will be the same objects in the input carboxysomes array, if provided, as objects are modified by reference) with its rubisco_pairs, num_events, ave_vector, S, eigenvalues, and S_val properties populated. For each rubisco in a Carboxysome, this script will also compute its orientation vector. The rubisco_pairs property will be filled with Rubisco_Pair objects (see Rubisco_Pair class) for each pair of rubisco in the carboxysome that passed the set distance threshold (2*√3*rubisco_diameter). The num_events property is the count of inner rubisco pairs that are within 2*rubisco_diameter distance and within 25 degrees of alignment angle. The ave_vector property is an average of the orientation vectors of all the rubisco within a Carboxysome. The S property represents the second order orientation tensor. The eigenvector property holds the eigenvectors of the 'S' alignment tensor. The S_val property is calculated as 3/2∗max(eigenvalues).

- **Helper Functions Used**
  - `compute_vector(rubisco)` `% Computes vector for a rubisco orientation`
  - `x_rot_matrix(theta)` `% Calculate the rotation matrix about the x-axis for a rotation of degrees specified by theta.`
  - `z_rot_matrix(theta)` `% Calculate the rotation matrix about the z-axis for a rotation of degrees specified by theta.`
  - `calc_angle(vec1, vec2)` `% Calculates angle in degrees between vectors vec1 and vec2 in 3D space.`

- `calc_distance(rubisco1, rubisco2) % Calculates the Euclidean norm between two Rubiscos rub1 and rub2.`
- `calc_projection(rubisco1, rubisco2) % Calculates the minimum projection between two Rubiscos rub1 and rub2.`
- `calc_S(rubisco) % Computes S-alignment tensor for a Rubisco rubisco.`
- `op(carb, inner_only) % Computes average S tensor for a Carboxysome carb. Accepts a Boolean inner_only to specify whether to only include inner Rubiscos or all.`
- `calc_average_vector(carb, inner_only) % Compute the average vector for a Carboxysome carb. Accepts a Boolean inner_only to specify whether to only include inner Rubiscos or all.`

3. linkages

Compute the linkages between rubisco in each carboxysome. A valid linkage is defined as a pair of rubisco which satisfy the following 3 conditions for possible binding: 1. The vertical distance from the center of the source rubisco to the projection of the target rubisco's center onto the source's axis is between *min_distance* and *max_distance*; 2. The angle between the axes of alignment of the two rubisco is less than *max_angle*; 3. The radial distance from the center of the target rubisco to the closest point in the source rubisco's main axis is within a distance of *search_radius*.

- **Syntax**

```
  carboxysome_data = linkages(filename, carboxysome_data,
min_distance_factor, max_distance_factor, radius_factor, max_angle,
mutual)
  carboxysome_data = linkages(filename, carboxysome_data)
  carboxysome_data = linkages(filename)
```

- **Description**

`carboxysome_data = linkages(filename, carboxysome_data, min_distance_factor, max_distance_factor, radius_factor, max_angle, mutual)` takes in an array of Carboxysome objects with pre-computed pairs from **local_global_alignment** and all necessary threshold values to consider a valid linkage. *min_distance_factor* and *max_distance_factor* are multiples of a rubisco diameter to search vertically along the source rubisco axis (labelled 1 in the figure above). *radius_factor* is the multiple of a rubisco diameter to search horizontally outward from the source's axis (3 in the figure above). *max_angle* is the maximum angle of alignment permitted between the two rubisco's main axes. *mutual* is a Boolean toggle which indicates whether a pair should reference one another (pass the 3 linkage requirements in both directions of the pair). Returns an array of Carboxysomes with its *links* property populated with all valid linkages found.

`carboxysome_data = linkages(filename, carboxysome_data)` allows the user to run the program without specifying their own linkage conditions' threshold values. By default, the *min_distance_factor* will be set to 0.7947 of a rubisco diameter, the *max_distance_factor* will be set to 1.4933 of a rubisco diameter, and the *mutual* toggle will be set to true. For the rest of the parameters, the user will be prompted to choose from two sets of recommended parameters: "Tight" or "Pivot". "Tight" parameters are more restricting, at 0.31 of a rubisco diameter for the *radius_factor*,

and 25 degrees for the *max_angle*, while "Pivot" parameters are more inclusive, at 0.45 of a rubisco diameter for the *radius_factor* and 50 degrees for the *max_angle*. All these recommended parameter values were determined from histogram analysis on our datasets. For RuBisCo, we determined that 0.45 of a rubisco diameter is the maximum *radius_factor* that can be used in order to prevent multiple bindings at the same binding site. If a larger radius is used, the search might result in a source RuBisCo having multiple top or bottom binding partners that satisfy the linkage conditions. Linkages are created with the user's choice for the set of parameters.

*Note: if using a different protein, the appropriate analysis must be conducted to determine the maximum *radius_factor* that does not allow for multiple bindings at the same site. Otherwise valid particles may be discarded later in the pipeline.

`carboxysome_data = linkages(filename)` allows the user to provide only the *filename* as input and use this script as the starting point (without having to run other scripts first). If only one argument is provided, the script will call back to **local_global_alignment** with the provided *filename* argument and generate the prerequisite Carboxysome array before creating linkages.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysome_data`	Array of Carboxysome objects for which to create linkages. Carboxysome objects must be pre-populated with rubisco pairs that pass the distance threshold from **local_global_alignment**. Each Rubisco_Pair object must have pre-computed *distance*, *projection*, and *angle*; and each Rubisco object must have its *vector* property computed. It is intended that the user run **local_global_alignment** first and then run linkages with the obtained output as an argument. Alternatively, the user may run this script as the starting point by only providing the filename parameter, and the script will reference back to **local_global_alignment** to obtain the required information.	Yes
`min_distance_factor`	The minimum vertical distance along the source rubisco axis, in multiples of a rubisco diameter, to begin the search for binding partners.	Yes

`max_distance_factor`	The maximum vertical distance along the source rubisco axis, in multiples of a rubisco diameter, where we want to end the search for binding partners.	Yes
`radius_factor`	The maximum radial distance, in multiples of a rubisco diameter, to search horizontally outward from the source's axis for binding partners.	Yes
`max_angle`	The maximum angle (in degrees) between two RuBisCo's main axes of alignment to consider as a valid linkage.	Yes
`mutual`	A Boolean toggle which indicates whether a pair should reference one another (passes the 3 linkage requirements in both directions of the pair). Encouraged if *radius_factor* is larger than 0.4	Yes

- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects (these will be the same objects in the input *carboxysomes* array, if provided, as objects are modified by reference) with its linkages populated as a subset of all rubisco_pairs where each pair falls under the specified definition of a valid linkage.

- **Helper Functions Used**
  - `calc_projection(rubisco1, rubisco2)` `% To compute the projection of a pair in the inverse direction`

4. chain_maker

This is the function following linkage formation in the pipeline. It takes all viable linkages and uses a data structure called a disjoint set to find all rubiscos that are connected to one another, which are defined to be rubisco chains. The *min_chain_length* argument represents the minimum length required for a set of linked rubiscos to be considered a chain and is set to a default of 3. After building the chains the rubiscos within them are put in order, storing which rubiscos are above and below them in the chain. If no linkage exists between two consecutive rubiscos, there is likely a bad data point. The particle with the lowest cc value is deleted, and a table with bad particles removed is saved to the working directory.

- **Syntax**

```
carboxysome_data = chain_maker(filename, carboxysome_data, min_cha
in_length)
```

```
carboxysome_data = chain_maker(filename, carboxysome_data)
carboxysome_data = chain_maker(filename)
```

- **Description**

`carboxysome_data = chain_maker(filename, carboxysome_data, min_chain_length)` takes in an array of Carboxysome objects with pre-computed properties from **linkages** as *carboxysome_data* and the threshold value to consider a valid chain. *min_chain_length* is a number describing the minimum number of rubiscos in a chain for a chain to be considered as valid. Returns an array of Carboxysomes with its *chains* property populated with all valid chains found. Each created chain is also assigned a *type* of ChainType which can be either free, attached, or edge to dictate whether a chain is on the edge, just attached to the shell, or free floating within the carboxysome.

`carboxysome_data = chain_maker(filename, carboxysome_data)` allows the user to run the program without specifying their own minimum chain length threshold value. By default, the *min_chain_length* will be set to 3 rubiscos. Chains are then generated with the default length as a minimum requirement.

`carboxysome_data = chain_maker(filename)` allows the user to provide only the *filename* as input and use this script as the starting point (without having to run other scripts first). If only one argument is provided, the script will call back to **linkages** with the provided *filename* argument and generate the pre-required Carboxysome array with linkage data before creating chains.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysome_data`	Array of Carboxysome objects for which to create chains. Carboxysome objects must be pre-populated with linkages where each pair is a valid linkage as computed by **linkages**. It is intended that the user run **linkages** first and then run **chain_maker** with the obtained output as an argument. Alternatively, the user may run this script as the starting point by only providing the filename parameter, and the script will reference back to **linkages** to obtain the required information.	Yes
`min_chain_length`	The minimum number of rubiscos linked together to be considered as a valid chain.	Yes

- **Output Arguments**

Argument	Description

carboxysome_data	An array of Carboxysome objects (these will be the same objects in the input *carboxysomes* array, if provided, as objects are modified by reference) with its chains populated as a set of Rubisco_Chain objects with properties of carboxysome_index, id, tag, tags, chain, and type.

- **Helper Functions Used**
  - `fill_data(carb, chain, table) % Fills in missing data after chains are formed`
  - `calc_S(rubisco) % Computes the S-alignment tensor for a rubisco`
  - `op(carb, chain) % Computes average S tensor for a rubisco chain`
  - `calc_average_vector(carb, chain) % Computes the average vector for a rubisco chain`
  - `calc_center(carb, chain) % Computes the center for a rubisco chain`
  - `find_furthest(carb, chain) % Locates the rubisco that is the furthest from the centroid`
  - `find_order(carb, chain, point) % Orders the rubiscos based on how far they are from the furthest rubisco to the centroid, since that should be an end rubisco`
  - `calc_angles(chain) % Finds the angle that the chain's average vector forms with the xy and z planes`
  - `align_rubisco_axes_in_chains(carb, chain, orig_table) % Makes above/below designations and catches errors`
  - `delete_problems(problem_particles, carb, orig_table) % Deletes problem particles with the lowest cc values`

5. chain_linkages

This function determines which chains are linked together based on their distance apart and the angle between them. Pairs of chains that are found to form a linkage are saved in a property of the carboxysome they belong to called "chain_links". Chains which participate in at least 6 chain links are called central chains.

- **Syntax**

```
  carboxysome_data = chain_linkages(filename, carboxysome_data, min_
distance, max_distance, max_angle, factor)
  carboxysome_data = chain_linkages(filename, carboxysome_data)
  carboxysome_data = chain_linkages(filename)
```

- **Description**

`carboxysome_data = chain_linkages(filename, carboxysome_data, min_distance, max_distance, max_angle, factor)` takes in an array of Carboxysome objects with precomputed properties from **chain_maker** as *carboxysome_data* and the threshold values for considering a valid chain linkage. *min_distance* is the minimum distance in nanometers required for a pair of chains to be considered linked. *max_distance* is the maximum required distance in nanometers

for a pair of chains to be considered a linkage. *max_angle* is the maximum possible angle in degrees between the two chains for them to be considered linked. *factor* is the minimum required overlap between two chains to be considered a linkage. Returns an array of Carboxysomes with the *chain_links* property populated with all valid chain links found.

`carboxysome_data = chain_linkages(filename, carboxysome_data)` allows the user to run the program without specifying the threshold values for considering a valid linkage. By default *min_distance* will be set to 10 nm, *max_distance* will be set to 14 nm, and *max_angle* set to 22 degrees.

`carboxysome_data = chain_linkages(filename)` allows the user to provide only the filename as input and use this script as the starting point (without having to run other scripts first). If only one argument is provided, the script will call back to **chain_maker** with the provided filename argument and generate the pre-required Carboxysome array with *chain* data before creating chain links.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysome_data`	Array of Carboxysome objects for which to create chain links. Carboxysome objects must be pre-populated with *chains* where each Rubisco_Chain is a valid *chain* as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **chain_linkages** with the obtained output as an argument. Alternatively, the user may run this script as the starting point by only providing the filename parameter, and the script will reference back to **chain_maker** to obtain the required information.	Yes
`min_distance`	The minimum distance in nanometers between two chains for a pair to be considered as a valid chain_link.	Yes
`max_distance`	The maximum distance in nanometers between two chains for a pair to be considered as a valid chain_link.	Yes
`max_angle`	The maximum angle in degrees between two chains for a pair to be considered as a valid chain_link.	Yes
`factor`	The minimum amount of overlap between chains needed to make a linkage (expressed as	Yes

	a decimal, e.g. 0.3). Can take values between 0 and 1.	

- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects (these will be the same objects in the input *carboxysomes* array, if provided, as objects are modified by reference) with its *chain_links* populated as a set of Chain_Pair objects with properties of carboxysome index, the two chain indices, the two chain tags, angle between chains, distance between chains, concentration of carboxysome, and gap.

- **Helper Functions Used**
  - `chain_distance_closest(chain_i, chain_j, carb, rubisco_diameter, factor) % Returns the distance between two chains, and the angle between two chains. Calculates the distance by breaking up each chain into straight segments and finding the minimum of segment-to-segment distances.`
  - `get_rubisco_positions(rubisco_tags, carbox) % Fetches the xyz positions of each rubisco in a chain and stores them in an array.`
  - `generate_segments(rubisco_positions, carbox, rubisco_tags, rubisco_diameter) % Creates the segments that make up a chain. For a chain with N rubiscos there are N+1 segments. We make N-1 interior segments between rubisco and two exterior segments which extend from an end-of-chain rubisco center to its edge.`
  - `closest_point_on_segment_to_point(free_point, endpoints) % Calculates the point on a line segment which minimizes the distance to a free point. This is done by calculating t, the ratio of the scalar projection (dot product) of the point on the segment divided by the length of the segment.`
  - `find_overlap(i_chain_segments, j_chain_segments) % finds the overlap between the chains by finding how far each endpoint of one chain extends along the other chain. The maximum overlap value calculated is used`
  - `findEnds(i_segments, j_segments) % finds the locations of the chain endpoints`

6. lattice_gen

This is the final function in the pipeline. Takes in the chain linkages and turns them into viable lattices based on the same principles as **chain_maker**. Chain groups whose size exceeds a user-defined threshold are considered a lattice. Carboxysome objects are populated with the chains which make up its lattice(s) and the type of lattice it contains.

- **Syntax**

```
  carboxysome_data = lattice_gen(filename, carboxysome_data,
threshold)
  carboxysome_data = lattice_gen(filename, carboxysome_data)
  carboxysome_data = lattice_gen(filename)
```

- **Description**

`carboxysome_data = lattice_gen(filename, carboxysome_data, threshold)` takes in an array of Carboxysome data with precomputed properties from **chain_linkages** *as carboxysome_data.* Returns an array of Carboxysomes with the *lattice* property populated with all valid lattices found, as well as the *lattice_type* property set to multiple, shell, none, full, or incomplete, depending upon characteristics of the lattice(s). The input variable *threshold* is the minimum number of a chains a group of chains must have to be considered a lattice.

`carboxysome_data = lattice_gen(filename, carboxysome_data)` allows the user to run the program without specifying the *threshold* variable. It will default to a value of 5.

`carboxysome_data = lattice_gen(filename)` allows the user to provide only the filename as input and use this script as the starting point (without having to run other scripts). If only one argument is provided, the script will call back to **chain_linkages** with the provided filename argument and generate the pre-required Carboxysome array with *chain_links* data before creating lattices.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysome_data`	Array of Carboxysome objects for which to create chain links. Carboxysome objects must be pre-populated with *chain_links* where each Chain_Pair is a valid *chain_link* as computed by **chain_linkages**. It is intended that the user run **chain_linkages** first and then run **lattice_gen** with the obtained output as an argument. Alternatively, the user may run this script as the starting point by only providing the filename parameter, and the script will reference back to **chain_linkages** to obtain the required information.	Yes
`threshold`	The minimum number of chains that a group of connected chains must have to be considered a lattice; default value of 5.	Yes

- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects (these will be the same objects in the input *carboxysomes* array, if provided, as objects are modified by reference) with its *lattice* property populated as a set of Rubisco_Chain objects and its lattice_type property populated with a LatticeType.

- **Helper Functions Used**
  - `find_graph(graph, nodes) % generates the graphical representation and finds the maximum number of connections to any one node`

7. main

This function runs the main pipeline automatically, prompting the user for input data along the way. After running **chain_maker** it allows the user to start over with the new table that was generated. The user can use default values or their own throughout the script.

- **Syntax**

```
carboxysome_data = main()
```

- **Description**

`carboxysome_data = main()` takes in no input. Instead of command line variable input, the user is prompted each time they need to enter a value. Returns an array of Carboxysomes with all properties populated as they would have been by running through the pipeline step by step.

- **Input Arguments**

None

- **Output Arguments**

Argument	Description
`carboxysome_data`	An array of Carboxysome objects with its properties populated as if the whole pipeline was taken step by step.

- **Helper Functions Used**

- `preprocess_datasets(varargin) % Prepares a .tbl file to be turned into Rubisco objects by cleaning data, concatenating odd and even files, and writing a new, processed .tbl file.`
- `contants(pixel_size, diameter) % Sets the pixel size and particle diameter to be global constants throughout the run of the pipeline.`
- `carb_volumes = convex_hull_and_volume(filename) % runs the first step in the pipeline`
- `carb_local = local_global_alignment(filename, carb_volumes) % runs the second step in the pipeline`
- `carb_linkages = linkages(filename, carb_local) % runs the third step in the pipeline`
- `carb_chains = chain_maker(filename, carb_linkages) % runs the fourth step in the pipeline`
- `carb_chain_links = chain_linkages(filename, carb_chains) % runs the fifth step in the pipeline`
- `carboxysome_data = lattice_gen(filename, carb_chain_links) % runs the sixth and final step in the pipeline`

8. concentration_volume_graphs

This function creates two plots. First, it creates a scatter plot of the number of outer rubiscos each carboxysome has as a function of its volume. The data of this plot are colored according to the total rubisco concentration of the carboxysome. The second scatter plot plots number of outer rubiscos as a function of total rubisco concentration. It fetches from the output data of **convex_hull_and_volume** the number of outer rubiscos, volume, and total rubisco concentration of each carboxysome.

- **Syntax**

```
concentration_volume_graphs(carboxysome_data)
```

- **Description**

`concentration_volume_graphs(carboxysome_data)` takes in an array of Carboxysome data with precomputed properties from **convex_hull_and_volume** *as carboxysome_data.* It retrieves the values of the "volume", "num_rubisco_outer", and "concentration" properties from each carboxysome. The first plot has "num_rubisco_outer" on the y axis, "volume" on the x axis, and "concentration" determines the color of each point. The second plot has "num_rubisco_outer" on the y axis and "concentration" on the x axis.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated	No

	with "rubisco" as computed by **convex_hull_and_volume**. It is intended that the user run **convex_hull_and_volume** first and then run **concentration_volume_graphs** with the obtained output as an argument.	

- **Output Arguments**

None

- **Helper Functions Used**

None

9. angle_with_outer_shell

   This function plots a histogram of the angles between outer rubiscos and the carboxysome shell. It fetches from the output data of **local_global_alignment** the orientation and average normal vector of each rubisco and calculates the angle between them. In addition, random angles are calculated and overlaid on the plot for comparison.

- **Syntax**

```
angle_with_outer_shell(carboxysome_data, bin_width)
```

- **Description**

`angle_with_outer_shell(carboxysome_data, bin_width)` takes in an array of Carboxysome data with precomputed properties from **local_global_alignment** *as carboxysome_data.* It retrieves the values of the "vector" and "average_normal" properties from each outer rubisco and calculates the angle between them. These angles are plotted as a histogram between 0 and 90 degrees. The width (in degrees) of each histogram bin is controlled by the user through the *bin_width* argument. Each entry on the histogram is given a color based on the total rubisco concentration of the carboxysome the rubisco belongs to. A group of random angles is calculated and overlaid on the plot in red to compare with the real data.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "rubisco_pairs" as computed by **local_global_alignment**. It is intended that the user run **local_global_alignment** first and then run **angle_with_outer_shell** with the obtained output as an argument.	No

`bin_width`	The width to make each bin in the plot, expressed in degrees. The full range of the plot is 0 to 90 degrees.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `Angle = calc_angle(vec1, vec2) % calculate the angle between two vectors in R^3`
  - `Mat = compute_random_vectors_marsaglia_72(N) % calculates N vectors distributed randomly on a sphere for the purpose of calculating the angle they make with some reference vector`

10. tensor_analysis_graphs

This function makes two scatter plots of the scalar orientation parameter S: one uses total rubisco concentration, the other uses inner rubisco concentration. It fetches from the output data of **local_global_alignment** the global scalar orientation parameter, total rubisco concentration, and inner rubisco concentration of each carboxysome.

- **Syntax**

```
tensor_analysis_graphs(carboxysomes, min_num_rubiscos)
```

- **Description**

`tensor_analysis_graphs(carboxysomes, min_num_rubiscos)` takes in an array of Carboxysome data with precomputed properties from **local_global_alignment** *as carboxysomes.* It retrieves the values of the "S", "concentration", and "inner_concentration" properties from each carboxysome. It first creates a scatter plot with "S" on the y axis and "concentration" on the x axis. Then it creates a scatter plot with "S" on the y axis and "inner_concentration" on the x axis. The user can filter out sparse carboxysomes by setting the *min_num_rubiscos* argument. Carboxysomes with fewer rubiscos than *min_num_rubiscos* will not be plotted.

- **Input Arguments**

Argument	Description	Optional
`carboxysomes`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "rubisco_pairs" as computed by **local_global_alignment**. It is intended that the user run **local_global_alignment** first and then run **tensor_analysis_graphs** with the obtained output as an argument.	No

`min_num_rubiscos`	The minimum number of rubiscos a carboxysome should contain to be plotted on these graphs.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

11. neighboring_rubisco_alignment_analysis

This function plots a series of probability density graphs of the angles between the orientations of neighboring rubiscos. It fetches from the output data of **local_global_alignment** the angles and distances between rubisco pairs, then filters out pairs based on user inputs. In addition, random angle data are calculated and overlaid on the plot for comparison.

- **Syntax**

```
  neighboring_rubisco_alignment_analysis(carboxysome_data, max_distanc
e, plot_only_inner, min_rubiscos_inner)
```

- **Description**

`neighboring_rubisco_alignment_analysis(carboxysome_data, max_distance, plot_only_inner, min_rubiscos_inner)` takes in an array of Carboxysome data with precomputed properties from **local_global_alignment** *as carboxysome_data.* It retrieves the values of the "angle", "distance" and "concentration" properties from each Rubisco_Pair. Rubisco_Pairs with a "distance" property less than *max_distance* will not be plotted. If the user sets *plot_only_inner* to true, then only Rubisco_Pairs comprised of two inner rubiscos will be plotted. Carboxysomes with fewer than *min_rubiscos_inner* inner rubiscos will not be plotted. A graph is made for each carboxysome and they are overlaid on top of each other. Each graph is a normal kernel probability density estimation with a bandwidth chosen by the user. Each graph is colored according to the carboxysome's total rubisco concentration. A group of random angles is calculated and overlaid on the plot in red to compare with the real data.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "rubisco_pairs" as computed by **local_global_alignment**. It is intended that the user run **local_global_alignment** first and then run **neighboring_rubisco_alignment_analysis** with the obtained output as an argument.	No

`max_distance`	The maximum distance to allow between rubiscos whose angle difference gets plotted (in pixels).	No
`plot_only_inner`	A Boolean that, when true, plots only Rubisco_Pairs made of two inner rubiscos.	No
`min_rubiscos_inner`	The minimum number of inner rubiscos a carboxysome should have to be plotted.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `angle = calc_angle(vec1, vec2) % calculate the angle between two vectors in R^3`
  - `mat = compute_random_vectors_marsaglia_72(N) % calculates N vectors distributed randomly on a sphere for the purpose of calculating the angle they make with some reference vector`

12. link_arc_analysis

This function creates a scatterplot of the spatial relationships between the Rubisco_Pair objects from **local_global_alignment**. It fetches from the output data of **local_global_alignment** the angles and distances between rubisco pairs, then filters out pairs based on user inputs. The resulting plot can be analyzed to pick appropriate input values for the next script in the main pipeline (**linkages**).

- **Syntax**

```
link_arc_analysis(carboxysome_data, label, pointSize, max_angle)
```

- **Description**

`link_arc_analysis(carboxysome_data, label, pointSize, max_angle)` takes in an array of Carboxysome data with precomputed properties from **local_global_alignment** as *carboxysome_data.* It retrieves the values of the "angle", "distance" and "projection" properties from each Rubisco_Pair. Rubisco_Pairs with an "angle" property greater than or equal to *max_angle* will not be plotted. If the user passes *label* some text, then the plot's title will include that text. Data points on the graph will have a value dictated by *pointSize*. The distance between rubisco in the radial direction is calculated using the "distance" property, "projection" property, and the Pythagorean theorem. The graph has radial distance on the x-axis and "projection" distance on the y-axis. Both axes' units are expressed as multiples of the particle diameter. Each data point is colored according to the value of its "angle" property.

- **Input Arguments**

Argument	Description	Optional

`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "rubisco_pairs" as computed by **local_global_alignment**. It is intended that the user run **local_global_alignment** first and then run **neighboring_rubisco_alignment_analysis** with the obtained output as an argument.	No
`label`	A string included at the end of the title of the plot to distinguish it from other plots (empty by default).	Yes
`pointSize`	The size of the plotted data points (defaults to 2).	Yes
`max_angle`	The maximum angle between a pair of rubisco that will still be plotted.	Yes

- **Output Arguments**

None

- **Helper Functions Used**

None

13. rubisco_alignment_analysis

This function creates a scatter plot of the number of aligned rubiscos in a carboxysome as a function of carboxysome characteristics. It fetches from the output data of **local_global_alignment** the number of Rubisco_Pairs that satisfy multiple filters. This number is plotted as a function of inner rubisco concentration.

- **Syntax**

```
rubisco_alignment_analysis(carboxysome_data, bin_width)
```

- **Description**

`angle_with_outer_shell(carboxysome_data, max_angle, max_distance, min_inner_conc)` takes in an array of Carboxysome data with precomputed properties from **local_global_alignment** *as carboxysome_data.* It retrieves the number of Rubisco_Pairs in each carboxysome that pass all filters. To pass all filters, the Rubisco_Pair must be comprised of only inner rubiscos, have a "distance" property less than *max_distance,* and have an "angle" property less than *max_angle.* Their parent carboxysome must also have an "inner_concentration" property of at least *min_inner_conc.* The size of points on the scatter plot corresponds to the carboxysome's "volume" property, and the colors of the points correspond to its "concentration" property.

- **Input Arguments**

Argument	Description	Optional

`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "rubisco_pairs" as computed by **local_global_alignment**. It is intended that the user run **local_global_alignment** first and then run **rubisco_alignment_analysis** with the obtained output as an argument.	No
`max_angle`	The maximum allowable angle between rubiscos to count as aligned (in degrees).	No
`max_distance`	The maximum allowable distance between rubiscos to count as aligned (in pixels).	No
`Min_inner_conc`	The minimum inner concentration a carboxysome must have to be plotted (in micro molar).	No

- **Output Arguments**

None

- **Helper Functions Used**

None

14. affinity_by_sites

This function creates two plots, an exponential fit, and a .csv file. This function starts by running **chain_maker** with a minimum chain length of 1 to set up its analysis. The first plot places the ratio of bound to unbound sites on rubisco on the y axis against total rubisco concentration (one plot point per carboxysome) on the x axis. The second plot places the ratio of bound to unbound sites against the inner rubisco concentration. It fits the data of the first plot to an exponential function and prints the results to the matlab command widow. Finally, it writes a .csv file containing information about the number of free and occupied binding sites in each carboxysome. To run this script accurately, the user's data must be through at least **linkages**.

- **Syntax**

```
  affinity_by_sites(filename, carboxysome_data,
label, min_chain_length)
```

- **Description**

`affinity_by_sites(filename, carboxysome_data, label, min_chain_length)` takes in an array of Carboxysome objects with precomputed properties from **linkages** *as carboxysome_data.* It uses that data and the string in *filename* to run **chain_maker** with a minimum chain length of 1 for this specific analysis. The function will plot the ratio of bound rubisco sites to unbound rubisco sites in two different ways with the *label* string to identify unique aspects of the plot of the user's preference. All rubiscos will be considered in the first plot, but only those in chains as long as *min_chain_length* will be considered in the second plot. Both plots have a

logarithmic y axis, and the second plot contains carboxysome volume information in its point sizes and ratio of rubsicos participating in chains information in its point colors.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "links" where each Rubisco_Pair is a valid linkage as computed by **linkages**. It is intended that the user run **linkages** first and then run **affinity_by_sites** with the obtained output as an argument.	No
`label`	A word that will be printed as part of the title of the plots. It can be used to distinguish plots with different data.	No
`min_chain_length`	In order to be a part of the second plot, rubiscos must participate in a chain of at least this length.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `carboxysome_data = chain_maker(filename, carboxysome_data) % creates a chain from every linkage, allowing the number of bound and unbound rubisco binding sites to be calculated`

15. visualize_carboxysome_3d

This function calls a javascript file to create an interactive model of a carboxysome. It opens a local website where the user can inspect the carboxysome model. It writes a file where the javascript file can find it and load it as a model.

- **Syntax**

```
visualize_carboxysome_3d(filename)
```

- **Description**

`Visualize_carboxysome_3d(filename)` takes the name of a .tbl file *filename* as input. If the file indicated by *filename* does not exist where the javascript file can find it, it writes the file where it can be found and tells the user where it is. Next it launches a local webpage to model a carboxysome. The user is presented with three prompts. On the first the user gives the location of

the file they want to load to model the carboxysome. On the second, the user gives the number of the carboxysome to model. On the third, the user gives the tags of any rubiscos to highlight on the model. For more information, see the file /Organization_and_Polymerization_of_Rubisco_in_Alpha_Carboxysomes/graphing_and_analysis_scripts/Visualize_Carboxysome_3D/Instructions.pdf.

- **Input Arguments**

Argument	Description	Optional
`filename`	A .tbl file whose data should be loaded to model the carboxysome.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

16. display_chain_tags

This function prints to the matlab command window a list of rubisco tags grouped into chains. It fetches from the output data of **chain_maker** the lists of tags for every chain in a carboxysome. The format of the output is designed to be copied and pasted into the **Visualize_Carboxysome_3D** third prompt.

- **Syntax**

```
display_chain_tags(carb_chains, carb_index)
```

- **Description**

`display_chain_tags(carb_chains, carb_index)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** *as carb_chains.* It retrieves the Rubisco_Chain objects from the Carboxysome object whose "carb_index" property is equal to *carb_index.* It prints the rubisco tags which make each chain, found in the "tags" property of the Rubisco_Chain. Each chain is enclosed in square brackets. If there is more than one chain, the chains are separated by commas and all enclosed in another set of square brackets. This formatting matches the expected input format of chains to highlight in the third prompt of the **Visualize_Carboxysome_3D** script, and can be copied and pasted into that field.

- **Input Arguments**

Argument	Description	Optional
`carb_chains`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as	No

	computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **display_chain_tags** with the obtained output as an argument.	
`carb_index`	The number of the carboxysome whose chains you want to print.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

17. neighboring_rubisco_bend_analysis

This function plots a histogram of the bend angles between adjacent rubiscos in chains. It fetches from the output data of **chain_maker** the Rubisco objects that are adjacent in chains and calculates the bend angle between them. In addition, the entries on the histogram are colored by the inner concentration of the parent carboxysome.

- **Syntax**

```
  neighboring_rubisco_bend_analysis(carboxysome_data, bin_limit, bin_w
idth, min_chain_length)
```

- **Description**

`Neighboring_rubisco_bend_analysis(carboxysome_data, bin_limit, bin_wid th, min_chain_length)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** *as carboxysome_data.* It retrieves two Rubisco objects at a time that are adjacent to each other in their chain. If the rubiscos belong to a chain whose "length" property is less than *min_chain_length* then they are not plotted. Otherwise the bend angle, the angle between the orientation vectors of the rubiscos, is calculated and plotted. These angles are plotted as a histogram between 0 and *bin_limit* degrees. The width (in degrees) of each histogram bin is controlled by the user through the *bin_width* argument. Each entry on the histogram is given a color based on the inner rubisco concentration of the carboxysome the chain belongs to.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **neighboring_rubisco_bend_analysis** with the obtained output as an argument.	No

`bin_limit`	The maximum angle (in degrees) to plot, recommended to be the same as the *max_angle* argument used as an input to **linkages.**	No
`bin_width`	The width to make each bin in the plot, expressed in degrees. The full range of the plot is 0 to *bin_limit* degrees.	No
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still have its rubiscos included in this analysis.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `bend = calc_bend(rubisco_i, rubisco_j) % calculate the bend angle between two rubiscos`

18. neighboring_rubisco_twist_analysis

This function plots a histogram of the twist angles between adjacent rubiscos in chains. It fetches from the output data of **chain_maker** the Rubisco objects that are adjacent in chains and calculates the twist angle between them. In addition, the entries on the histogram are colored by the inner concentration of the parent carboxysome.

- **Syntax**

```
neighboring_rubisco_twist_analysis(carboxysome_data, bin_width, min_chain_length)
```

- **Description**

`Neighboring_rubisco_twist_analysis(carboxysome_data, bin_width, min_chain_length)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** *as carboxysome_data.* It retrieves two Rubisco objects at a time that are adjacent to each other in their chain. If the rubiscos belong to a chain whose "length" property is less than *min_chain_length* then they are not plotted. Otherwise the twist angle, the angle between the narot angles of the rubiscos, is calculated and plotted. These angles are plotted as a histogram between -45 and +45 degrees. The width (in degrees) of each histogram bin is controlled by the user through the *bin_width* argument. Each entry on the histogram is given a color based on the inner rubisco concentration of the carboxysome the chain belongs to.

- **Input Arguments**

Argument	Description	Optional

`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **neighboring_rubisco_twist_analysis** with the obtained output as an argument.	No
`bin_width`	The width to make each bin in the plot, expressed in degrees. The full range of the plot is -45 to +45 degrees.	No
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still have its rubiscos included in this analysis.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `twist = calc_twist(rubisco_i, rubisco_j, chain.average_vector) % calculate the twist angle between two rubiscos`

19. twist_and_bend_analysis_graph

This function makes a scatter plot of the twist and bend angles between adjacent rubiscos in chains. It fetches from the output data of **chain_maker** the Rubisco objects that are adjacent in chains and calculates the bend and twist angles between them. In addition, the entries on the histogram are colored by the inner concentration of the parent carboxysome.

- **Syntax**

```
twist_and_bend_analysis_graph(carboxysome_data, min_chain_length)
```

- **Description**

`twist_and_bend_analysis_graph(carboxysome_data, min_chain_length)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** as *carboxysome_data.* It retrieves two Rubisco objects at a time that are adjacent to each other in their chain. If the rubiscos belong to a chain whose "length" property is less than *min_chain_length* then they are not plotted. Otherwise the twist angle is plotted on the y axis and the bend angle is plotted on the x axis. Each data point is given a color based on the inner rubisco concentration of the carboxysome the chain belongs to.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated	No

	with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **twist_and_bend_analysis_graph** with the obtained output as an argument.	
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still have its rubiscos included in this analysis.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `bend = calc_bend(rubisco_i, rubisco_j) % calculate the bend angle between two rubiscos`
  - `twist = calc_twist(rubisco_I, rubisco_j, chain.average_vector) % calculate the twist angle between two rubiscos`

20. rubisco_spacing_analysis

This function plots a series of probability density graphs of the distances between adjacent rubiscos in a chain, one for each carboxysome. It fetches from the output data of **chain_maker** the "projection" values of Rubisco_Pair objects, grouped by parent carboxysome. Each graph is colored according to its carboxysome's inner concentration.

- **Syntax**

```
rubisco_spacing_analysis(carboxysome_data, min_chain_length, min_conc)
```

- **Description**

`rubisco_spacing_analysis(carboxysome_data, min_chain_length, min_conc)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** *as carboxysome_data.* It retrieves the value of the "projection" property from each Rubisco_Pair object in a chain. The distances are grouped by their parent carboxysome and made into separate overlaid plots. These plots are all normal kernel density estimations with a bandwidth chosen by the user. Rubisco_Pair objects in chains shorter than *min_chain_length* are omitted from the plot, as are Rubisco_Pair objects in carboxysomes with inner concentrations less than min_conc. Each plot is colored according to the inner concentration of the carboxysome it represents.

- **Input Arguments**

Argument	Description	Optional

`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **rubisco_spacing_analysis** with the obtained output as an argument.	No
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still have its rubiscos included in this analysis.	No
`min_conc`	The minimum inner concentration (expressed in micro molar) a carboxysome must have to be included in this analysis.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

21. chain_lengths_analysis

This function plots a histogram of the lengths of rubisco chains. It fetches from the output data of **chain_maker** the "length" properties of Rubisco_Chain objects. The user can set requirements that chains must meet in order to be plotted. In addition, the entries on the histogram are colored by the total rubisco concentration of the parent carboxysome.

- **Syntax**

```
chain_lengths_analysis(carboxysome_data, min_chain_length, min_linka
ges)
```

- **Description**

`Chain_lengths_analysis(carboxysome_data, min_chain_length, min_linkage s)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** as *carboxysome_data.* It retrieves the lengths of chains in the data. A chain whose "length" property is less than *min_chain_length* is not plotted. A chain participating in fewer Chain_Pair objects than *min_linkages* will not be plotted. In order to use the *min_linkages* filter appropriately, the data must contain properties computed from **chain_linkages.** Setting *min_linkages* to zero will allow the use of data with precomputed properties from **chain_maker**. Each data point is given a color based on the total rubisco concentration of the carboxysome the chain belongs to.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **chain_lengths_analysis** with the obtained output as an argument.	No
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still be included in this analysis.	No
`min_linkages`	The minimum number of Chain_Pair objects a chain must participate in to still be included in this analysis. Only for use after data population from **chain_linkages.** To run this script before that point, enter zero for this value.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

22. chain_spacing_analysis

This function makes two plots: a histogram of the angles between neighboring chains, and a set of overlaid probability density functions of the distances between chains. It fetches from the output data of **chain_maker** the Rubisco_Chain objects that pass user-specified filters and calculates the distances and angles between them. In addition, the entries on both plots are colored by the inner concentration of the parent carboxysome.

- **Syntax**

```
chain_spacing_analysis(carboxysome_data, min_chain_length, max_dista
nce, bin_width)
```

- **Description**

`chain_spacing_analysis(carboxysome_data, min_chain_length, max_distanc e, bin_width)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** as *carboxysome_data.* It retrieves two Rubisco_Chain objects at a time from the same carboxysome. Rubisco_Chain objects with a "length" property less than *min_chain_length* are skipped. The distance and angle between the chains are calculated. The probability density function plot groups the data by parent carboxysome and stacks the plots on one figure. The pdf is a normal kernel density estimation with a bandwidth chosen by the user. The histogram data is filtered further, only plotting data points if the calculated distance is less than *max_distance.* The width (in degrees) of each histogram bin is controlled by the user through the *bin_width* argument. The data for both plots are given a color based on the inner rubisco concentration of the carboxysome the chains belong to.

- **Input Arguments**

Argument	Description	Optional
`carboxysome_data`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "chains" as computed by **chain_maker**. It is intended that the user run **chain_maker** first and then run **chain_spacing_analysis** with the obtained output as an argument.	No
`min_chain_length`	The minimum length (expressed in number of rubiscos) a chain can be to still be included in this analysis.	No
`max_distance`	The maximum distance (in nanometers) chains can have between them to still have the angle between them plotted in the histogram.	No
`bin_width`	The width to make each bin in the plot, expressed in degrees. The full range of the plot is 0 to 90 degrees.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `[distance, angle] = chain_distance_closest(chain_i, chain_j, carboxysome, rubisco_diameter) % calculate the distance and angle between two chains`
  - `rubisco_positions = get_rubisco_positions(rubisco_tags, carboxysome) % get an array containing the positions of the rubiscos indicated by rubisco_tags`
  - `chain_segments = generate_segments(rubisco_positions, carboxysome, rubisco_tags, rubisco_diameter) % break up a chain into segments connecting rubiscos and create an array of those segments`
  - `closest_point = closest_point_on_segment_to_point(free_point, endpoints) % calculate the location of the closest spot on a line segment to a free-floating point`

23. synthetic_data_generation

This script generates two tables of synthetic data for validation purposes. 1) A particle table with randomized coordinates and euler angles. 2) A particle table with a known percent of particles that form linkages.

- **Syntax**

```
synthetic_data_generation()
```

- **Description**

`synthetic_data_generation()` takes a table of particles as a reference to generate two tables of synthetic data to validate the results of the main script. The first table will be a particle table with random coordinates and euler angles. This table will demonstrate that filaments in the real data are based on real linkages. The second table will be a particle table with a known seeded number of filaments based on maximum parameters for distance, bend, and twist to test how well the main script identifies the linkages.

- **Input Arguments**

None, the script will prompt the user for input

- **Output Arguments**

None

- **Helper Functions Used**

None

24. display_lattice_tags

This function prints to the matlab command window a list of rubisco tags grouped into chains. It fetches from the output data of **lattice_gen** the lists of tags for every lattice in a carboxysome. The format of the output is designed to be copied and pasted into the **Visualize_Carboxysome_3D** third prompt.

- **Syntax**

```
display_lattice_tags(carb_chains, carb_index)
```

- **Description**

`display_lattice_tags(carb_chains, carb_index)` takes in an array of Carboxysome data with precomputed properties from **lattice_gen** as *carb_chains.* It retrieves the Rubisco_Chain objects from the "lattice" property of the Carboxysome object whose "carb_index" property is equal to *carb_index.* It prints the rubisco tags which make each lattice. Each chain is enclosed in square brackets. If there is more than one chain, the chains are separated by commas and all enclosed in another set of square brackets. This formatting matches the expected input format of chains to highlight in the third prompt of the **Visualize_Carboxysome_3D** script, and can be copied and pasted into that field.

- **Input Arguments**

Argument	Description	Optional
`carb_chains`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "lattice" as computed by **lattice_gen**. It is intended that the user run **lattice_gen** first and then run **display_lattice_tags** with the obtained output as an argument.	No

`carb_index`	The number of the carboxysome whose lattice you want to print.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

25. create_class_table

This function writes a .csv file containing information about every rubisco in the dataset. It fetches from the output data of **lattice_gen** a variety of information pertaining to rubiscos being in chains. If applicable, it calculates the twist and bend angles to adjacent rubiscos and the shortest distance to a linked chain.

- **Syntax**

```
create_class_table(carboxysomes, outfile)
```

- **Description**

`create_class_table(carboxysomes, outfile)` takes in an array of Carboxysome data with precomputed properties from **chain_maker** as *carboxysomes*. For each Rubisco object in the dataset it retrieves its "index" property, "reg" property, its parent carboxysome's volume, carboxysome's total rubisco concentration, carboxysome's inner rubisco concentration, "inside" property, whether it is in a chain, whether its chain is central to a lattice, a list of its oligomerization partners, and distances to its partners in the chain. Furthermore, it calculates the twist angles to its partners in the chain, bend angles to its partners in the chain, and the shortest distance to a linked chain. Each value is stored in a column and each Rubisco is stored in a row. The data is written to a .csv file whose name is given by the user in the *outfile* argument.

- **Input Arguments**

Argument	Description	Optional
`carboxysomes`	Array of Carboxysome objects to conduct analysis on. Carboxysome objects must be pre-populated with "lattice" as computed by **lattice_gen**. It is intended that the user run **lattice_gen** first and then run **create_class_table** with the obtained output as an argument.	No
`outfile`	The name of a .csv file to write the data to. Must end with .csv.	No

- **Output Arguments**

None

- **Helper Functions Used**
  - `bend = calc_bend(rubisco_i, rubisco_j) % calculate the bend angle between two rubiscos`
  - `twist = calc_twist(rubisco_I, rubisco_j, chain_lead_vector) % calculate the twist angle between two rubiscos`

26. preprocess_datasets

Prepare the data for analysis by cleaning it, ensuring it is in bin 2, combining it into one file if necessary, and checking it for invalid entries (like NaN and Inf). Write the prepared data to a .tbl file with a name chosen by the user.

- **Syntax**

```
  preprocess_datasets(filename, outfile, clean_value, is_bin2)
  preprocess_datasets(oddfile, evenfile, outfile, clean_value,
is_bin2)
```

- **Description**

`preprocess_datasets(filename, outfile, clean_value, is_bin2)` takes in a .tbl file, a name for an output file, a clean value, and whether the data is in bin 2. The data in the input .tbl file is cleaned by *clean_value* without adjusting the bin size. Next it adds the xyz shifts from columns 4-6 of the input data to the xyz values in columns 24-26 and zeroes the xyz shifts columns. If *is_bin2* is false, the data is converted from bin 1 to bin 2. The data is checked for NaN and Inf values, then written as a .tbl file according to the name in *outfile*.

`preprocess_datasets(oddfile, evenfile, outfile, clean_value, is_bin2)` takes in two .tbl files, a name for an output file, a clean value, and whether the data is in bin 2. The data in the input .tbl files are cleaned separately by *clean_value* without adjusting the bin size, then concatenated into one array. Next it adds the xyz shifts from columns 4-6 of the data to the xyz values in columns 24-26 and zeroes the xyz shifts columns. If *is_bin2* is false, the data is converted from bin 1 to bin 2. The data is checked for NaN and Inf values, then written as a .tbl file according to the name in *outfile*.

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	Yes

`oddfile`	Input file with half the particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	Yes
`evenfile`	Input file with half the particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	Yes
`outfile`	The name of a file to write the processed data to. Must be a file with '.tbl' extension.	No
`clean_value`	Clean size to apply to your data. Cleaning will occur in whatever bin size your input data is in.	No
`is_bin2`	A boolean value describing whether your data is in bin 2. If false, your data is assumed to be in bin1, and will be converted to bin 2 for the outfile.	No

- **Output Arguments**

None

- **Helper Functions Used**

None

27. read_data_tbl

Open and read a .tbl file into an array in the matlab workspace.

- **Syntax**

```
omdTable = read_data_tbl(filename)
```

- **Description**

`omdTable = read_data_tbl(filename)` takes in the .tbl file indicated by "filename" and uses Dynamo's dread() command to load it into the matlab workspace as an array called "omdTable". It returns the array "omdTable".

- **Input Arguments**

Argument	Description	Optional
`filename`	Input file with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section. Must be a file with '.tbl' extension.	No

- **Output Arguments**

Argument	Description
`omdTable`	An array of data with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section.

- **Helper Functions Used**

None

28. get_carboxysome_indices_from_tomogram

Get a list of all the carboxysomes in a dataset.

- **Syntax**

```
carb_indices = get_carboxysome_indices_from_tomogram(data)
```

- **Description**

`carb_indices = get_carboxysome_indices_from_tomogram(data)` takes in a *data* array matching the Dynamo format. It finds all the unique values of column 21 (carboxysome id column) and stores them in the list *carb_indices*. It returns the list *carb_indices*.

- **Input Arguments**

Argument	Description	Optional
`data`	An array of data with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section.	No

- **Output Arguments**

Argument	Description
`carb_indices`	A list of all the different carboxysome indices present in the dataset.

- **Helper Functions Used**

None

29. read_rubisco_objects_from_tomogram

Create a Rubisco object for each row of an array of data in Dynamo format.

- **Syntax**

```
rubiscos = read_rubisco_objects_from_tomogram(data)
```

- **Description**

`rubiscos = read_rubisco_objects_from_tomogram(data)` takes in a *data* array matching the Dynamo format. It uses each row of the array to create an instance of a Rubisco object. The Rubisco objects are saved in an array called *rubiscos.* The array *rubiscos* is returned.

- **Input Arguments**

Argument	Description	Optional
`data`	An array of data with particle data matching Dynamo format. One row per particle, 38 columns with fields as described in Input Data Format section.	No

- **Output Arguments**

Argument	Description
`rubiscos`	An array of Rubisco objects whose values were populated by the values in each row of the input data. One Rubisco object per row in the input data.

- **Helper Functions Used**

None

30. Carboxysome

Model for a Carboxysome. Contains properties and functions relating to processing. Contains information about the carboxysome itself and the rubisco, chains, and lattice inside it.

- **Properties**

Property	Input Type	Description
`carb_index`	Double	Index of Carboxysome.

rubisco	Rubisco	List of Rubisco objects in the Carboxysome.
num_rubisco	Double	Number of Rubiscos in the Carboxysome.
num_rubisco_inner	Double	Number of Rubiscos inside the boundary of the convex hull.
num_rubisco_outer	Double	Number of Rubiscos on the boundary of the convex hull.
tags_inside	Double	Tags of the Rubisco inside the boundary.
convex_hull	Double	Array of outer rubisco indices which make up triangular faces of the Carboxysome boundary.
vertices	Double	Coordinates of the vertices of the convex hull. Each row is a vector corresponding to the position of the vertex in R^3.
coordinates	Double	The xyz coordinates of every rubisco in the Carboxysome.
volume	Double	Volume of the Carboxysome (in cubic meters).
concentration	Double	Total Rubisco concentration in the Carboxysome (in micro molar).
inner_concentration	Double	Concentration of inner Rubisco in the Carboxysome (in micro molar).
num_events	Double	Number of inner Rubisco pairs in the Carboxysome which are neighbors and well aligned.
rubisco_pairs	Rubisco_Pair	List of Rubisco_Pair objects that are neighbors in the Carboxysome.
normals	Double	Array of vectors normal to the faces of the convex hull.
centroids	Double	Array of xyz positions of the centroids of the faces of the convex hull.
S	Double	Tensor describing global orientation of Rubisco in the Carboxysome.
eigenvalues	Double	Eigenvalues of the S tensor.
S_val	Double	Scalar parameter describing global orientation of Rubisco in the Carboxysome.
ave_vector	Double	Average of all inner Rubisco orientation vectors.

links	Rubisco_Pair	List of Rubisco_Pair objects that are oligomerization partners.
chains	Rubisco_Chain	List of Rubisco_Chain objects in the Carboxysome.
chain_links	Chain_Pair	List of Chain_Pair objects in the Carboxysome.
lattice	Rubisco_Chain	Array of chain lattices in the Carboxysome, represented by their constituent Rubisco_Chain objects.
max_connections	Double	Maximum number of Chain_Pairs a single chain makes in the Carboxysome.
tomo	Double	Tomogram index the Carboxysome belongs to.
lattice_type	LatticeType	The type of lattice he Carboxysome has.

- **Methods**
  - `carb = Carboxysome(index, rubiscos, num_rubiscos, num_rubiscos_inner, num_rubiscos_outer, tags_in, conv_hull, vol, verts, cords, conc, inner_conc, event, norms, cents, s, ave, pairs, links) % Constructor: creates an instance of a Carboxysome`
  - `disp(self) % Displays the properties of this Carboxysome`
  - `obj_copy = copy(obj) % Creates a copy of this object with all properties.`
  - `coords = get_coords(obj) % Calculates the centroid of a face of the convex hull using the vertices surrounding it.`
  - `visualize(self) % Visualize the convex hull of the carboxysome.`
  - `visualize_normals(self) % Visualize the convex hull of the carboxysome with normal vectors on its faces.`

31. Rubisco

Model for a Rubisco. Contains properties and functions relating to processing. Contains information about the carboxysome itself and everything inside it. Specifically used as input type for the *rubisco* property of the Carboxysome class.

- **Properties**

Property	Input Type	Description
tag	Double	Unique identifier for the Rubisco in the whole dataset.

index	Double	Unique identifier for the Rubisco within its parent Carboxysome.
aligned	Boolean	Whether this Rubisco underwent alignment.
averaged	Boolean	Whether this Rubisco was included in subtomogram averaging.
dx	Double	X-direction offset from subtomogram center.
dy	Double	Y-direction offset from subtomogram center.
dz	Double	Z-direction offset from subtomogram center.
tdrot	Double	Euler angle rotation about z (in degrees).
tilt	Double	Euler angle rotation about new x (in degrees).
narot	Double	Euler angle rotation about new z (in degrees).
cc	Double	Cross-correlation coefficient.
cc2	Double	Cross-correlation coefficient after thresholding 2.
cpu	Double	Processor that aligned the particle.
ftype	Double	Fourier sampling type.
ymintilt	Double	Minimum angle in tilt series around y axis.
ymaxtilt	Double	Maximum angle in tilt series around y axis.
xmintit	Double	Minimum angle in tilt series around x axis.
xmaxtilt	Double	Maximum angle in tilt series around x axis.
fs1	Double	Free parameter for Fourier sampling #1.
fs2	Double	Free parameter for Fourier sampling #2.
tomo	Double	Tomogram number this Rubisco belongs to.
reg	Double	Carboxysome number this Rubisco belongs to.
class	Double	Class number.
annotation	Double	Field for arbitrary labels.
x	Double	X position of Rubisco (in pixels).
y	Double	Y position of Rubisco (in pixels).
z	Double	Z position of Rubisco (in pixels).
dshift	Double	Norm of shift vector.
daxis	Double	Difference in axis orientation.
dnarot	Double	Difference in narot.

dcc	Double	Difference in cc.
otag	Double	Original tag from subboxing.
npar	Double	Number of added particles / subunit label.
ref	Double	Reference used in multireference projects.
sref	Double	Subreference.
apix	Double	Angstroms per pixel.
def	Double	Defocus (in microns).
eig1	Double	Eigencoefficient #1.
eig2	Double	Eigencoefficient #2.
ave_normal	Double	Average of normal vectors of surrounding convex hull faces (only if outer Rubisco).
vector	Double	Orientation vector of Rubisco.
inside	Boolean	Whether the Rubisco is an inner Rubisco.
rubisco_above_me	Double	Tag of the Rubisco above this one in the chain.
rubisco_below_me	Double	Tag of the Rubisco below this one in the chain.
in_central_chain	Boolean	Whether the rubisco's chain is on the interior of a lattice.

- **Methods**
  - `this_rubisco = Rubisco(tag1, aligned1, averaged1, dx1, dy1, dz1, tdrot1, tilt1, narot1, cc1, cc21, cpu1, ftype1, ymintilt1, ymaxtilt1, xmintilt1, xmaxtilt1, fs11, fs21, tomo1, reg1, class1, annotation1, x1, y1, z1, dshift1, daxis1, dnarot1, dcc1, otag1, npar1, ref1, sref1, apix1, def1, eig11, eig21, ave_norm, vec) % Constructor: creates an instance of a Rubisco`
  - `disp(self) % Displays the properties of this Rubisco to the matlab command window`
  - `obj_copy = copy(obj) % Creates a copy of this object with all properties.`

32. Rubisco_Pair

Model for a pair of Rubiscos. One Rubisco is designated as the "i" Rubisco, and the other is the "j" Rubisco. Contains properties and functions relating to processing. Specifically used as input type for the *rubisco_pairs* property of the Carboxysome class, and used in the scripts **local_global_alignment** and **linkages.**

- **Properties**

Property	Input Type	Description
reg	Double	Index of parent Carboxysome.
num_rubisco	Double	Number of Rubiscos in parent Carboxysome.
num_rubisco_inner	Double	Number of inner Rubiscos in parent Carboxysome.
volume	Double	Volume of parent Carboxysome (in cubic meters).
inner	Boolean	If both Rubiscos are inner Rubiscos.
I_index	Double	Index of the i Rubisco.
J_index	Double	Index of the j Rubisco.
I_tag	Double	Tag of the i Rubisco.
J_tag	Double	Tag of the j Rubisco.
angle	Double	Angle in degrees between the two Rubiscos.
distance	Double	Distance between the two Rubisco centers (in pixels).
concentration	Double	Concentration of the Carboxysome. (micro molar).
inner_concentration	Double	Inner concentration of the Carboxysome (micro molar).
projection	Double	Projection of vector pointing from source rubisco to target rubisco onto source rubisco's orientation vector.

- **Methods**
  - `pair = Rubisco_Pair(carb_index, num_rubisco, num_rubisco_inner, vol, in, I_index, J_index, I_tag, J_tag, ang, dist, conc, inner_conc) % Constructor: creates an instance of a Rubisco Pair`
  - `disp(self) % Displays the properties of this Rubisco Pair object`
  - `obj_copy() = copy(obj) % Creates a copy of this object with all properties.`

33. DisjointSet

Data structure used to separate a set into groups with no overlapping members. Contains properties and functions relating to determining group members and sizes. Specifically used to create Rubisco_Chain objects and lattices for the *lattice* property of the Carboxysome class. Used in the scripts **chain_maker** and **lattice_gen.**

- **Properties**

Property	Input Type	Description
`parent`	Double	List of each element's parent.
`rank`	Double	List of each element's number of children.
`size`	Double	Number of elements in the set.
`all`	Double	List of the elements in the set.

- **Methods**
  - `set = DisjointSet(numElements) % Constructor: creates an instance of a DisjointSet with a given size`
  - `root = locate(set, x) % Locates and returns the most fundamental parent (root) of element x`
  - `unite(set, x, y) % Unites the elements x and y, including their children`
  - `children = find_children(set, parent) % Locates and collects all children of the parent into the array "children"`
  - `Chains = chain_builder(set) % Puts grouped elements into unique entries in a cell array`

34. Rubisco_Chain

Model for a chain of Rubiscos. Contains properties and functions relating for processing. Specifically used as input type for the *chains* property of carboxysome_data.

- **Properties**

Property	Input Type	Description
`length`	Double	The numeric length of the Rubisco Chain measured in Rubiscos.
`reg`	Double	Index of the Carboxysome containing this Rubisco Chain.
`index`	Double	Index of the Rubisco Chain, which uniquely identifies it within its parent Carboxysome.
`tag`	Double	Tag of the Rubisco Chain, which uniquely identifies it within the whole dataset.
`tags`	Double	Tags of the Rubiscos within the chain.
`indices`	Double	Indices of the Rubiscos within the chain.
`average_vector`	Double	The average vector for a Rubisco Chain, computed by averaging the orientations of the Rubiscos in the Rubisco Chain. See *calc_ave_vector* in **chain_maker.**

`centroid`	Double	Average position of the Rubiscos in the chain.
`eigenvalues`	Double	Eigenvalues of the S tensor of the chain.
`S`	Double	Average S tensor for a Rubisco Chain. See *op* in **chain_maker**.
`S_val`	Double	Scalar orientation parameter of the chain.
`type`	ChainType	The ChainType of the Rubisco Chain. Has a value of attached, edge, or free.
`xy_angle`	Double	Angle that the chain's average vector makes in the xy-plane (in degrees). See *calc_angles* in **chain_maker**.
`z_angle`	Double	Angle that the chain's average vector makes with the xy-plane (in degrees). See *calc_angles* in **chain_maker**.

- **Methods**
  - `chain = Rubisco_Chain(reg, index, tag, tags, indices, type) % Constructor: creates an instance of a Rubisco Chain`
  - `disp(self) % Displays the properties of this Rubisco_Chain object`
  - `obj_copy() = copy(obj) % Creates a copy of this object with all properties.`

35. calc_bend

Calculate the bend angle between adjacent rubiscos in a chain.

- **Syntax**

```
bend = calc_bend(rubisco1, rubisco2)
```

- **Description**

`bend = calc_bend(rubisco1, rubisco2)` takes in two Rubisco objects. It calculates the angle between their orientation vectors, called *bend* in degrees. Due to the symmetry of a rubisco, the range of values *bend* can take is 0 degrees to 90 degrees. The *bend* angle is returned.

- **Input Arguments**

Argument	Description	Optional
`rubisco1`	A Rubisco object which is adjacent to rubisco2 in a chain.	No

`rubisco2`	A Rubisco object which is adjacent to rubisco1 in a chain.	No

- **Output Arguments**

Argument	Description
`bend`	The angle between the orientation vectors of the two input Rubisco objects expressed in degrees.

- **Helper Functions Used**

None

36. calc_narot

Checks that a Rubisco object's "vector" property (its orientation vector) points in the same direction as the "average_vector" property of the Rubisco_Chain to which it belongs. This function does the first step to calculate the twist angle between adjacent rubiscos in a chain.

- **Syntax**

```
narot = calc_narot(rubisco, chain_lead_vector)
```

- **Description**

`narot = calc_narot(rubisco, chain_lead_vector)` takes in a Rubisco object and the average vector of the Rubisco_Chain to which it belongs. It determines if the Rubisco object's orientation vector points along the average vector of the Rubisco_Chain or against it. It returns the Rubisco object's "narot" property if they do point in the same direction. Otherwise, it returns the negative of the "narot" property. There is no restriction on the value of "narot".

- **Input Arguments**

Argument	Description	Optional
`rubisco`	A Rubisco object which is adjacent to another rubisco in a chain.	No
`chain_lead_vector`	The "average vector" property of the chain to which the Rubisco belongs.	No

- **Output Arguments**

Argument	Description
`narot`	The Euler angle of the Rubisco object which is its rotation around its new z axis expressed in degrees.

- **Helper Functions Used**

None

37. calc_twist

Calculates the twist angle between two consecutive rubiscos in a chain by finding the difference in their narot values. It takes advantage of the c4 symmetry of rubiscos to limit the possible values of the twist angle and expresses its answer in degrees.

- **Syntax**

```
twist = calc_twist(rubisco1, rubisco2, chain_lead_vector)
```

- **Description**

`twist = calc_twist(rubisco1, rubisco2, chain_lead_vector)` takes in two Rubisco objects and the average vector of the Rubisco_Chain to which they belongs. After calling **calc_narot** once for each Rubisco object, it subtracts their narot angles to get the difference, called *twist* angle. It uses modulus operations and the c4 symmetry of the rubisco to define the range of possible values *twist* can have. The *twist* is returned with a value between +45 degrees and –45 degrees.

- **Input Arguments**

Argument	Description	Optional
`rubisco1`	A Rubisco object which is adjacent to rubisco2 in a chain.	No
`rubisco2`	A Rubisco object which is adjacent to rubisco2 in a chain.	No
`chain_lead_vector`	The "average vector" property of the chain to which the Rubiscos belong.	No

- **Output Arguments**

Argument	Description
`twist`	The difference in narot angles between two adjacent rubiscos in a chain (in degrees).

- **Helper Functions Used**
  - `narot = calc_narot(rubisco, chain_lead_vector) % Ensures the narot angles are correct by checking the Rubisco orientation vector with the average vector of the chain it belongs to.`

38. ChainType

Model for the type of a Rubisco Chain. Contains properties and functions relating to assigning chain types. Specifically used to fill the *type* property of the Rubisco_Chain class. Used in the script **chain_maker.**

- **Possible Values**

Value	Description
`attached`	Chain contains 1 or 2 outer rubiscos.
`free`	Chain consists entirely of inner rubiscos.
`edge`	Chain contains 3 or more outer rubiscos.

- **Methods**
  - `disp(obj) % Displays the value of this ChainType object`
  - `str = to_str(obj) % Converts the ChainType value to a string`

39. Chain_Pair

Model for a pair of Rubisco_Chain objects. Contains properties and functions relating to processing. One Rubisco_Chain is designated as the "i" chain, and the other is the "j" chain. Specifically used as the input type for the *chain_links* property of the Carboxysome class, and used in the scripts **chain_linkages** and **lattice_gen**.

- **Properties**

Property	Input Type	Description
`reg`	Double	Index of the Carboxysome containing this Rubisco chain pair.
`I_index`	Double	Index of the i-chain.
`J_index`	Double	Index of the j-chain.
`I_tag`	Double	Tag of the i-chain.
`J_tag`	Double	Tag of the j-chain.

`angle`	Double	Angle between the chains' average vectors in degrees.
`distance`	Double	Distance between the chains in pixels.
`concentration`	Double	Concentration of the parent Carboxysome (in micro molar).

- **Methods**
  - `pair = Chain_Pair(carb_index, I_index, J_index, I_tag, J_tag, ang, dist, conc) % Constructor: creates an instance of a Chain Pair`
  - `disp(self) % Displays the properties of this Chain Pair object`
  - `obj_copy() = copy(obj) % Creates a copy of this object with all properties.`

40. LatticeType

Model for the type of a chain lattice. Contains properties and functions relating to assigning lattice types. Specifically used to fill the *lattice_type* property of the Carboxysome class. Used in the script **lattice_gen.**

- **Possible Values**

Value	Description
`full`	At least one chain in the lattice makes 6 or more Chain_Pair objects.
`incomplete`	Lattice has at least 4 chains and does not fit any other assignment.
`multiple`	Carboxysome contains more than one lattice.
`shell`	Over half of the carboxysome's chains are edge type.
`none`	Carboxysome has no lattice.

- **Methods**
  - `disp(obj) % Displays the value of this LatticeType object`
  - `str = to_str(obj) % Converts the LatticeType value to a string`

41. constants

Contains constant values for use throughout the pipeline. Constant values include particle diameter in meters, pixel size in meters, and Avogadro's number. Returns a CONSTANTS variable with properties containing the constant values.

- **Syntax**

```
CONSTANTS = constants(pixel_size, diameter)
CONSTANTS = constants()
```

- **Description**

`CONSTANTS = constants(pixel_size, diameter)` takes in the pixel size of the source tomograms in meters per pixel and the diameter of the particle to be analyzed in meters. The global constants PIXEL_SIZE and RUBISCO_DIAMETER_M are set to these values, respectively. The Avogadro's number constant is automatically set. It returns these values in the CONSTANTS variable.

`CONSTANTS = constants()` takes no input. If the first syntax has been used, it returns the constant values set by the most recent usage of that syntax. If this is the first call to constants(), it will set and return the default values of 2.54e-10 for pixel_size and 9.936e-9 for particle diameter.

- **Input Arguments**

Argument	Description	Optional
`pixel_size`	The pixel size of the tomogram sourcing the data expressed in meters per pixel.	Yes
`diameter`	The diameter of the particle of interest, expressed in meters.	Yes

- **Output Arguments**

Argument	Description
`CONSTANTS`	A variable that holds three constant values: pixel size, particle diameter, and Avogdro's number.

- **Helper Functions Used**

None